

Code Optimization (30 points)

1) Increment Y (5 points)

```
;Y <- Y + 1
LDI R16, 1
ADD YL, R16
LDI R16, 0
ADC YH, R16
```

```
ADIW Y, 1
```

2) Set interrupt flag and return (3 points)

```
;set interrupt flag and return
SEI
RET
```

```
RETI
```

3) Absolute value of R16 (7 points)

Given: R16 - 8-bit signed value

Result: R16 - absolute value of the original value in R16 (R16 if R16 $\geq$ 0 and -R16 if R16 $<$ 0)

Code: can be done in 5 instructions

```
MOV R17, R16 -copy over r16
LSL R17      -get MSB
SBC R17,R17  -create mask for MSB operations
EOR R16,R17  -invert r16 if it was negative
SUB R16,R17  -add the 1 if carry was negative
```

4) Count number of set bits in R17 | R16 (15 points)

Given: R17|R16 - 16-bit unsigned value

Result: R16 - count of number of set bits in R17|R16 (value will be between 0 and 16)

Code: can be done in 26 instructions

```
;acquire count through pairs
MOV R19, R17
MOV R18, R16
LSR R19          ;shift right lsb to carry
ROR R18          ;shift right carry to msb
ANDI R19, 0x55   ;apply 01010101 mask to each
ANDI R18, 0x55   ;0101 -> 0101
SUB R16, R18     ;1011 - 0101 = 01 | 10
                ;01 = 1 bit in high, 10 = 2 bits in low
SUB R17, R19     ;repeat process here as well

;sum up pair counts
MOV R19,R17
MOV R18,R16

;combined count method x = (x & 0x3333) + ((x >> 2) & 0x3333))
;I.E: 01|10 = 1 + 2 can add by them by:
;shifting top over, delete top half from both, add together
;0010(x&0x3) + 0001((x >> 2) & 0x3333) = 0011 = 3

LSR R19
ROR R18
LSR R19
ROR R18
ANDI R19, 0x33 ; Mask
ANDI R18, 0x33
ANDI R17, 0x33
ANDI R16, 0x33
ADD R17, R19 ; Add pairs, each contains bit count
ADD R16, R18

;repeat process with larger mask (swap instead of shifting)
ADD R16, R17 ; add corresponding halves together
MOV R18, R16
SWAP R18 ; swap instead of shiting
ANDI R16, 0x0F ; same mask as last time
ANDI R18, 0x0F
ADD R16, R18 ;updated final count
```